پروژه پایانترم محاسبات عددی

شماره دانشجویی: 403110409

الف)

کد مربوط به این بخش: `project1.cu`

- روش CPU: در این روش از سه حلقه تو در تو با پیچیدگی زمانی $O(N^3)$ برای محاسبه ضرب ماتریس‌ها استفاده شده است. تابع `cpu_matrixMultiplication` این محاسبات را انجام می‌دهد و نتایج آن در ادامه به عنوان مرجع برای مقایسه و بررسی صحت خروجی‌های روش‌های مبتنی بر GPU استفاده می‌شود.
- روش GPU Simple: در این هسته `gpu_simple_matrixMultiplication`، هر Thread محاسبه یک درایه از ماتریس خروجی را انجام می‌دهد. در هر مرحله از محاسبه، داده‌های موردنیاز مستقیم از حافظه Global خوانده می‌شوند و هیچ استفاده مجددی از داده‌ها صورت نمی‌گیرد. به همین دلیل، با وجود موازی‌سازی، تعداد زیاد دسترسی‌ها به حافظه Global که تأخیر بالایی دارد باعث کاهش کارایی این روش می‌شود.
- روش GPU Tiled: برای کاهش تعداد دسترسی‌ها به حافظه Global، هسته `gpu_tiled_matrixMultiplication` پیاده‌سازی شده است. در این روش ماتریس‌ها به بلوک‌های ۱۶ × ۱۶ تقسیم می‌شوند و هر بلوک از ماتریس‌های A و B تنها یک بار از حافظه Global در حافظه مشترک `__shared__` بارگذاری می‌شود. سپس تمامی هایThread بلوک پس از همگام‌سازی با دستور `__syncthreads()`، چندین بار از همین داده‌های موجود در حافظه مشترک استفاده می‌کنند. در نتیجه تعداد دسترسی به حافظه Global کاهش یافته، از داده‌ها دوباره استفاده می‌شود و زمان اجرای هسته نسبت به روش ساده کاهش می‌یابد.
- استفاده از کتابخانه cuBLAS: از تابع `cublasSgemm` برای نوع float و از تابع `cublasDgemm` برای نوع double استفاده شده است. این توابع پیاده‌سازی بسیار بهینه‌شده‌ای از ضرب ماتریس را در اختیار قرار می‌دهند و از بهینه‌سازی‌های داخلی GPU بهره می‌برند. برای نوع int، از آنجا که تابع استاندارد مستقیمی در cuBLAS برای int32 وجود ندارد، ابتدا ورودی‌ها به نوع int8_t تبدیل شده و سپس از تابع `cublasGemmEx` استفاده شده است. همچنین برای جلوگیری از Overflow در ضرب، نوع محاسبات داخلی و خروجی ۳۲ بیت (CUBLAS_COMPUTE_32I) است.

برای اندازه‌گیری زمان اجرای بخش‌های مختلف برنامه از رویدادهای CUDA یعنی `cudaEvent_t` استفاده شده است. زمان اجرای نسخه‌های GPU به سه بخش زیر تفکیک شده است:

۱. H2D: زمان انتقال ماتریس‌های A و B از حافظه اصلی (Host) به حافظه GPU (Device)

۲. Kernel Execution: زمان اجرای هسته روی GPU

۳. D2H: زمان انتقال ماتریس حاصل C از حافظه GPU به حافظه اصلی

زمان کل اجرا نیز از مجموع این سه بخش محاسبه و گزارش می‌شود.

همچنین پس از اجرای هر یک از روش‌های GPU، خروجی حاصل با خروجی نسخه CPU مقایسه می‌شود. این کار توسط تابع `check_result` انجام می‌شود. به دلیل تفاوت در ترتیب انجام جمع و ضرب در محاسبات ممیز شناور در پردازش موازی، برای مقادیر float از خطای مجاز 10^{-4} و برای مقادیر double از خطای مجاز 10^{-6} استفاده شده است تا اختلاف‌های ناشی از Round-off Error باعث گزارش نادرست نتیجه نشوند. برای نوع داده int هم برابری دقیق تمام درایه‌های ماتریس بررسی می‌شود.

ب)

کد مربوط به این بخش: `project2.cu`

در این بخش، ماتریس A به گونه‌ای تولید می‌شود که یک ماتریس sparse باشد یعنی بیشتر درایه‌های آن صفر باشند. در اینجا برای سادگی فرض شده است که حدود ۹۰ درصد درایه‌های ماتریس A صفر باشند. ماتریس B هم مانند بخش قبل Dense است. حاصل ضرب این دو ماتریس به صورت زیر محاسبه می‌شود:

$$C = \alpha AB + \beta C$$

برای ذخیره ماتریس A از قالب فشرده‌سازی سطری (CSR) استفاده شده است که شامل سه آرایه یک‌بعدی است:

- Values: آرایه‌ای به طول NNZ که مقادیر غیرصفر ماتریس را ذخیره می‌کند. در اینجا NNZ تعداد درایه‌های غیرصفر ماتریس است.
- ColIndices: آرایه‌ای به طول NNZ که اندیس ستون مربوط به هر درایه غیرصفر را نگهداری می‌کند.

- RowOffsets: آرایه‌ای به طول $N + ۱$ که محل شروع درایه‌های هر سطر را در دو آرایه Values و ColIndices مشخص می‌کند.

این قالب علاوه بر کاهش حافظه مصرفی، باعث می‌شود عملیات تنها روی درایه‌های غیرصفر انجام شده و از انجام محاسبات غیرضروری روی عناصر صفر جلوگیری شود.

برای انجام محاسبات روی GPU از کتابخانه cuSPARSE استفاده شده است. ابتدا با استفاده از تابع `cusparseCreateCsr` ماتریس A به صورت یک ماتریس sparse تعریف می‌شود. سپس ماتریس‌های B و C توسط تابع `cusparseCreateDnMat` به صورت ماتریس‌های dense تعریف می‌شوند. از آنجا که الگوریتم‌های cuSPARSE برای اجرای عملیات ضرب به حافظه موقت نیاز دارند، اندازه این حافظه با استفاده از تابع `cusparseSpMM_bufferSize` محاسبه شده و سپس با `cudaMalloc` اختصاص داده می‌شود. در ادامه تابع `cusparseSpMM` عملیات SpMM، یعنی ضرب یک ماتریس sparse در یک ماتریس dense را روی GPU انجام می‌دهد. ضرایب برابر با $\alpha = ۱$ و $\beta = ۰$ انتخاب شده‌اند تا رابطه به $C = AB$ تبدیل شود. مشابه بخش قبل، زمان انتقال داده از Host به Device (H2D)، زمان اجرای کرنل و زمان انتقال نتیجه از Device به Host (D2H) با استفاده از `cudaEvent` اندازه‌گیری شده و زمان کل اجرا نیز از مجموع این سه بخش محاسبه و گزارش شده است.

ج)

در ادامه، هر جدول مربوط به یکی از روش‌های CPU، GPU Simple، GPU Tiled، cuBLAS و cuSPARSE برای هر نوع داده float، double و int رسم شده‌اند. در هر بار اجرا در هر جدول برابر ب N مقادیر مختلف ۳۲، ۶۴، ۱۲۸، ۲۵۶، ۵۱۲، ۱۰۲۴ و ۲۰۴۸ قرار داده شده است. خروجی‌های مربوط به اجرای کدها نیز به طور کامل در فایل `output.txt` آمده‌اند.

نوع داده float

جدول ۱: CPU برای float

N	Total Time (ms)
32	0.080
64	0.610
128	5.014
256	51.683
512	434.616
1024	3559.496
2048	152420.719

جدول ۲: GPU Simple برای float

N	H2D (ms)	Kernel (ms)	D2H (ms)	Total Time (ms)
32	0.245	0.132	0.200	0.576
64	0.073	0.101	0.082	0.256
128	0.283	0.237	0.393	0.912
256	0.520	0.729	0.491	1.740
512	1.250	5.151	0.935	7.336
1024	4.351	38.488	2.205	45.043
2048	13.813	380.215	7.502	401.529

جدول ۳: GPU Tiled برای float

N	H2D (ms)	Kernel (ms)	D2H (ms)	Total Time (ms)
32	0.245	0.174	0.294	0.713
64	0.073	0.058	0.078	0.209
128	0.283	0.197	0.300	0.780
256	0.520	0.312	0.379	1.211
512	1.250	2.196	0.785	4.231
1024	4.351	16.867	2.063	23.281
2048	13.813	118.651	7.265	139.728

جدول ۴: cuBLAS برای float

N	H2D (ms)	Kernel (ms)	D2H (ms)	Total Time (ms)
32	0.245	0.302	0.264	0.811
64	0.073	0.243	0.203	0.519
128	0.283	0.306	0.306	0.894
256	0.520	0.648	0.295	1.463
512	1.250	0.667	0.780	2.697
1024	4.351	2.819	1.924	9.094
2048	13.813	16.265	6.645	36.722

جدول ۵: cuSPARSE برای float

N	H2D (ms)	Kernel (ms)	D2H (ms)	Total Time (ms)
32	0.258	3.977	0.187	4.422
64	0.527	3.747	0.168	4.442
128	0.297	3.715	0.350	4.362
256	0.350	3.864	0.356	4.570
512	0.813	4.339	0.606	5.759
1024	2.136	6.661	1.635	10.431
2048	9.258	50.737	6.973	66.968

نوع داده double

جدول ۶: CPU برای double

N	Total Time (ms)
32	0.106
64	0.743
128	4.604
256	56.097
512	450.440
1024	17792.324
2048	176564.938

جدول ۷: GPU Simple برای double

N	H2D (ms)	Kernel (ms)	D2H (ms)	Total Time (ms)
32	0.250	0.012	0.224	0.486
64	0.176	0.028	0.178	0.382
128	0.310	0.275	0.315	0.899
256	0.877	1.072	0.656	2.606
512	2.430	7.993	1.276	11.699
1024	8.241	102.139	4.064	114.443
2048	27.723	540.686	16.212	584.621

جدول ۸: GPU Tiled برای double

N	H2D (ms)	Kernel (ms)	D2H (ms)	Total Time (ms)
32	0.250	0.070	0.102	0.422
64	0.176	0.180	0.196	0.552
128	0.310	0.242	0.178	0.730
256	0.877	1.665	0.533	3.075
512	2.430	11.772	1.083	15.285
1024	8.241	131.471	3.758	143.469
2048	27.723	612.020	13.653	653.396

جدول ۹: cuBLAS برای double

N	H2D (ms)	Kernel (ms)	D2H (ms)	Total Time (ms)
32	0.250	0.093	0.058	0.400
64	0.176	0.367	0.252	0.796
128	0.310	0.309	0.207	0.826
256	0.877	1.600	0.541	3.019
512	2.430	9.929	1.110	13.469
1024	8.241	65.685	3.600	77.526
2048	27.723	427.774	13.679	469.176

جدول ۱۰: cuSPARSE برای double

N	H2D (ms)	Kernel (ms)	D2H (ms)	Total Time (ms)
32	0.183	0.036	0.129	0.348
64	0.317	0.038	0.074	0.429
128	0.352	0.225	0.290	0.867
256	0.583	0.378	0.489	1.450
512	1.306	1.454	1.030	3.790
1024	4.134	9.564	3.441	17.138
2048	15.577	98.423	13.893	127.893

نوع داده int

جدول ۱۱: CPU برای int

N	Total Time (ms)
32	0.076
64	0.633
128	4.828
256	52.934
512	431.958
1024	3691.754
2048	148948.203

جدول ۱۲: GPU Simple برای int

N	H2D (ms)	Kernel (ms)	D2H (ms)	Total Time (ms)
32	0.068	0.012	0.060	0.140
64	0.083	0.018	0.109	0.210
128	0.176	0.074	0.229	0.479
256	0.558	0.685	0.500	1.743
512	1.599	5.016	1.026	7.641
1024	12.195	189.268	2.063	203.526
2048	13.558	384.153	7.824	405.535

جدول ۱۳: GPU Tiled برای int

N	H2D (ms)	Kernel (ms)	D2H (ms)	Total Time (ms)
32	0.068	0.043	0.055	0.165
64	0.083	0.051	0.136	0.270
128	0.176	0.097	0.232	0.505
256	0.558	0.495	0.446	1.499
512	1.599	2.021	0.825	4.446
1024	12.195	15.774	1.731	29.700
2048	13.558	115.885	7.161	136.604

جدول ۱۴: cuBLAS برای int

N	H2D (ms)	Kernel (ms)	D2H (ms)	Total Time (ms)
32	0.050	0.073	0.061	0.184
64	0.061	0.086	0.110	0.257
128	0.146	0.181	0.237	0.564
256	0.416	0.157	0.403	0.977
512	0.449	0.460	0.915	1.824
1024	1.093	0.767	1.846	3.706
2048	3.532	4.240	7.487	15.259

جدول ۱۵: cuSPARSE برای int

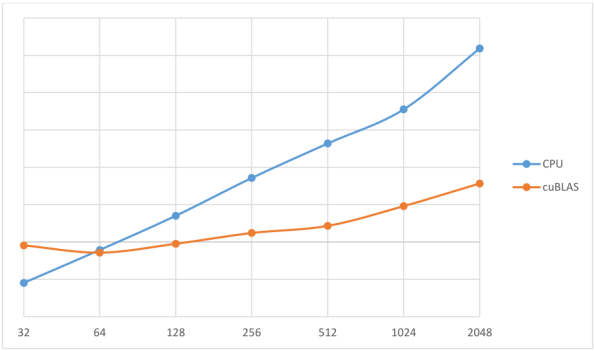
N	H2D (ms)	Kernel (ms)	D2H (ms)	Total Time (ms)
32	0.097	0.037	0.056	0.190
64	0.238	0.037	0.114	0.389
128	0.314	0.048	0.265	0.627
256	0.339	0.084	0.377	0.800
512	0.412	0.567	0.586	1.565
1024	1.141	3.308	1.806	6.254
2048	2.879	22.283	6.875	32.037

د)

جدول ۱۶: زمان کل cuBLAS و CPU برای نوع float

N	CPU	cuBLAS
32	0.080	0.811
64	0.610	0.519
128	5.014	0.894
256	51.683	1.463
512	434.616	2.697
1024	3559.496	9.094
2048	152420.719	36.722

شکل ۱: نمودار لگاریتمی مقایسه زمان کل float

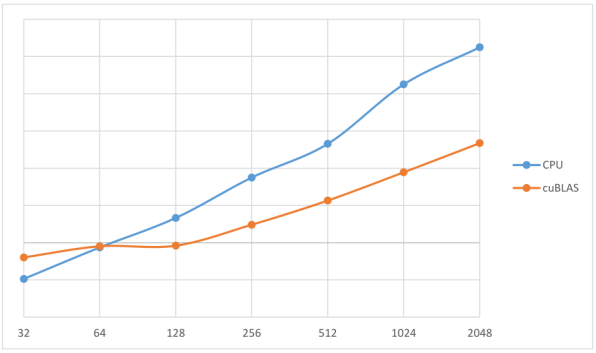


برای float، در ابعاد کوچک یعنی حدود $N = ۳۲$، CPU زمان کل کمتری نسبت به cuBLAS دارد. اما همان‌طور که در نمودار مشخص است از کمی قبل از $N = ۶۴$ به بعد، چون نمودار زمان CPU با شیب تندتری افزایش می‌یابد، زمان اجرای cuBLAS کمتر بوده و اختلاف این دو با افزایش N زیاد می‌شود.

جدول ۱۷ : زمان کل cuBLAS و CPU برای نوع double

N	CPU	cuBLAS
32	0.106	0.400
64	0.743	0.796
128	4.604	0.826
256	56.097	3.019
512	450.440	13.469
1024	17792.324	77.526
2048	176564.938	469.176

شکل ۲ : نمودار لگاریتمی مقایسه زمان کل double

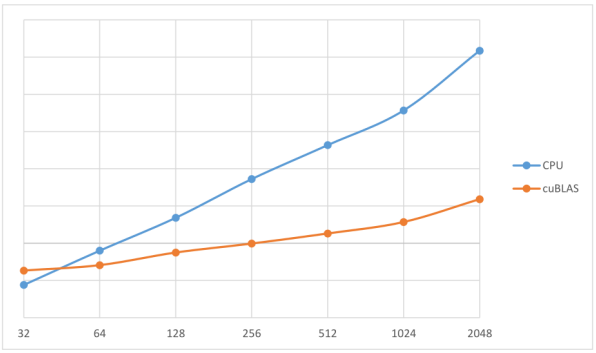


برای double، به دلیل دو برابر بودن حجم هر درایه نسبت به float، نقطه‌ای که روش GPU از CPU سریع‌تر می‌شود کمی دیرتر رخ می‌دهد. برای ابعاد $N = ۳۲$ و $N = ۶۴$، عملکرد CPU از نظر زمان کل بهتر است اما با توجه به محل تقاطع نمودارها از حدود N=۶۵ به بعد، نمودار cuBLAS پایین‌تر قرار گرفته و زمان اجرای GPU کمتر می‌شود.

جدول ۱۸ : زمان کل cuBLAS و CPU برای نوع int

N	CPU	cuBLAS
32	0.076	0.184
64	0.633	0.257
128	4.828	0.564
256	52.934	0.977
512	431.958	1.824
1024	3691.754	3.706
2048	148948.203	15.259

شکل ۳ : نمودار لگاریتمی مقایسه زمان کل int



برای int، تنها در بعد $N = ۳۲$ زمان اجرای CPU کمتر است و در $N = ۶۴$ به طور قابل توجهی بیشتر از GPU است. از حدود $N = ۴۵$ به بعد، cuBLAS زمان اجرای بسیار کمتر از CPU دارد

جدول زیر برای هر نوع داده مقدار تقریبی N را که از آن به بعد نسخه GPU سریع‌تر از نسخه CPU می‌شود، نشان می‌دهد:

جدول ۱۹: cuSPARSE برای float

type	N
float	63
double	65
int	45

دلیل اینکه در Nهای کوچک زمان اجرای نسخه CPU کمتر است، وجود Memory Transfer Overhead در روش GPU است. زمان کل اندازه‌گیری‌شده برای GPU علاوه بر زمان اجرای کرنل، شامل زمان انتقال داده‌ها یعنی H2D و D2H نیز می‌شود. در ابعاد کوچک، این سربار زمانی از زمان موردنیاز برای انجام محاسبات بیشتر است؛ بنابراین CPU سریع‌تر عمل می‌کند. اما با افزایش N، حجم محاسبات افزایش یافته و توان پردازش موازی GPU و پیاده‌سازی بهینه کتابخانه cuBLAS باعث می‌شود زمان اجرای GPU با نرخ کمتری افزایش یابد و در نتیجه برای ابعاد بزرگ‌تر از CPU سریع‌تر باشد.

۵)

مقایسه نسخه‌های مختلف: در نسخه GPU Simple، هر Thread یک درایه از ماتریس خروجی را محاسبه می‌کند و دسترسی‌های زیاد به حافظه Global باعث تاخیر بالایی می‌شوند که کارایی را محدود می‌کند. در نسخه GPU Tiled، با استفاده از حافظه مشترک (Shared Memory) که تاخیر کمتری دارد، داده‌های هر بلوک از ماتریس‌ها یک بار در حافظه مشترک بارگذاری شده و سپس توسط تمام هایThread آن بلوک مورد استفاده قرار می‌گیرند. این کار تعداد دسترسی‌ها به حافظه Global را کاهش داده و نسبت محاسبات به انتقال داده را بهبود می‌دهد، بنابراین سرعت اجرا نسبت به نسخه GPU Simple افزایش می‌یابد. کتابخانه cuBLAS نیز با استفاده از پیاده‌سازی‌های بسیار بهینه، از تکنیک‌هایی مانند Loop Unrolling، Prefetching و در معماری‌های جدید در صورت امکان از Tensor Cores استفاده می‌کند. همان‌طور که نتایج عددی نشان می‌دهند، در ابعاد بزرگ cuBLAS سریع‌ترین زمان اجرا را برای ماتریس‌های Dense دارد. برای ماتریس‌های Sparse نیز استفاده از فرمت CSR باعث می‌شود فقط درایه‌های غیرصفر پردازش شوند که حجم داده‌های انتقالی و تعداد عملیات را کاهش می‌دهد و در نتیجه زمان اجرای ضرب ماتریس نسبت به حالت Dense کمتر می‌شود.

تاثیر انواع داده: در داده‌های نوع float، به دلیل حجم کمتر، یعنی ۴ بایت برای هر درایه، انتقال داده‌ها با سرعت بیشتری انجام می‌شود. همچنین واحدهای پردازش ممیز شناور ۳۲ بیتی (FP32) بیشترین Throughput را دارند که باعث کمترین زمان اجرای هسته محاسباتی می‌شود. در داده‌های نوع double، اندازه هر درایه ۸ بایت است که حجم انتقال داده روی گذرگاه PCIe و همچنین حجم داده‌های جابه‌جا شده در حافظه GPU را دو برابر می‌کند و باعث افزایش زمان‌های H2D و D2H می‌شود. همچنین در کارت گرافیک استفاده‌شده، توان پردازشی واحدهای FP64 کمتر از FP32 است که باعث افزایش زمان اجرای هسته محاسباتی می‌شود. داده‌های نوع int از نظر حجم انتقال داده مشابه float هستند، اما در این پروژه برای اجرای cuBLAS ابتدا به نوع int8 تبدیل شده و سپس با استفاده از تابع `cublasGemmEx` با محاسبات داخلی ۳۲ بیتی پردازش شده‌اند. کوچک‌تر بودن اندازه داده‌های ورودی و بهینه‌سازی‌های موجود در پیاده‌سازی این تابع باعث شده است که در نتایج این پروژه، زمان اجرای آن از دو نوع داده دیگر کمتر باشد.

بخش دوم

ایده اول

در این روش با توجه به اینکه محاسبه هر درایه از ماتریس خروجی مستقل از سایر درایه‌ها است، ماتریس خروجی C را به دو بخش سطری مساوی یعنی نیمه بالایی و نیمه پایینی تقسیم می‌کنیم. از آنجایی که هر سطر از ماتریس C فقط به همان سطر از ماتریس A و کل ماتریس B وابسته است، ماتریس A را به صورت افقی به دو ماتریس A_1 و A_2 با ابعاد $\frac{N}{2} \times N$ تقسیم می‌کنیم. GPU اول ماتریس A_1 و کل ماتریس B را در حافظه Global خود دریافت کرده و نیمه بالایی خروجی یعنی $C_1 = A_1 B$ را محاسبه می‌کند. GPU دوم هم ماتریس A_2 و کل ماتریس B را دریافت کرده و نیمه پایینی خروجی یعنی $C_2 = A_2 B$ را محاسبه می‌کند. در نهایت C_1 و C_2 همزمان از هر دو GPU به حافظه Host منتقل شده و ماتریس نهایی C به دست می‌آید.

ایده دوم

هر دو ماتریس A و B را به صورت بلوکی به چهار ماتریس با ابعاد $\frac{N}{2} \times \frac{N}{2}$ تقسیم می‌کنیم:

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

GPU اول ردیف اول بلوک‌های خروجی یعنی C_{11} و C_{12} می‌شود و GPUدوم، ردیف دوم یعنیC_{21} و C_{22} را محاسبه می‌کند. برای این کار lrGPU اول برای محاسبه $C_{11} = A_{11}B_{11} + A_{12}B_{21}$، فقط به بلوک‌های A_{11}، A_{12}، B_{11} و B_{21} نیاز دارد. و به همین ترتیب GPU دوم هم فقط بخش‌های مربوط به خود را دریافت می‌کند.

مقایسه دو ایده

در ایده اول، کل ماتریس B باید در حافظه Global هر دو GPU به صورت تکراری ذخیره شود. اگر N بزرگ باشد ممکن است حافظه یک GPU برای ذخیره نصف ماتریس A، کل ماتریس B و نصف ماتریس C کافی نباشد. اما در ایده دوم، حافظه به شکل بهینه‌تری مدیریت می‌شود و هیچ داده‌ای به طور کامل کپی نمی‌شود، پس می‌توان ماتریس‌های بزرگ‌تری را پردازش کرد. در نتیجه ایده دوم از نظر مدیریت حافظه بهتر است.

ایده اول کاملا موازی و مستقل است و هیچ نیازی به ارتباط بین دو GPU در حین اجرای هسته‌ها وجود ندارد و پیاده‌سازی آن با CUDA Streamها ساده است. اما ایده دوم نیاز به مدیریت دقیق و زمان‌بندی ارتباطات بین دو GPU دارد. تاخیر ناشی از تبادل داده بین دو GPU می‌تواند باعث کاهش کارایی شود. در نتیجه ایده اول از نظر سادگی پیاده‌سازی و overhead ارتباطات بهتر است.

در نتیجه برای ماتریس‌هایی که در حافظه جا می‌شوند ایده اول به دلیل سادگی، کارایی و سرعت اجرای بهتری دارد اما برای ماتریس‌های بزرگ استفاده از ایده دوم به دلیل محدودیت‌های حافظه لازم است.